Fonctions en Swift - Partie 1

420-249-AH - Programmation mobile iOS

1. Pourquoi utiliser des fonctions ?

Une fonction permet de regrouper plusieurs instructions afin de réaliser une tâche précise.

Dans une application, une fonction peut servir à :

- Calculer un prix
- Valider un mot de passe
- Afficher un message
- Convertir une température
- Calculer une moyenne
- Vérifier un âge
- Préparer une donnée avant son affichage

Sans fonction, les mêmes instructions risquent d'être répétées plusieurs fois.

Exemple :

```swift
let prix = 20.0
let quantite = 3
let total = prix * Double(quantite)
print("Total : \(total) $")
```

Si ce calcul doit être effectué plusieurs fois, il est préférable de l'organiser dans une fonction.

2. Déclarer une fonction

En Swift, une fonction est déclarée avec :

```swift
func
```

Syntaxe générale :

```swift
func nomDeLaFonction() {
    // instructions
}
```

Exemple :

```swift
func afficherBienvenue() {
    print("Bienvenue dans l'application")
}
```

À ce moment, la fonction existe, mais elle n'est pas encore exécutée.

3. Appeler une fonction

Pour exécuter une fonction :

```
afficherBienvenue()
```

Exemple complet :

```
func afficherBienvenue() {
    print("Bienvenue dans l'application")
}

afficherBienvenue()
```

Résultat :

```
Console
Bienvenue dans l'application
```

4. Déclaration et appel

Il faut distinguer :

Déclaration

```
func afficherMenu() {
    print("Menu principal")
}
```

On définit ce que la fonction doit faire.

Appel

```
afficherMenu()
```

On demande à Swift d'exécuter la fonction.

5. Une fonction peut être appelée plusieurs fois

```
func afficherNotification() {
    print("Nouvelle notification")
}

afficherNotification()
afficherNotification()
afficherNotification()
```

Résultat :

```
Console
Nouvelle notification
Nouvelle notification
Nouvelle notification
```

L'intérêt est de ne pas répéter les mêmes instructions.

6. Une fonction peut contenir plusieurs instructions

```
func afficherEtatApplication() {
    print("----------------")
    print("Application active")
    print("Connexion disponible")
    print("----------------")
}
```

Puis :

```
afficherEtatApplication()
```

Une fonction représente généralement une responsabilité précise.

7. Donner un nom clair à une fonction

Un bon nom indique l'action effectuée.

Préférable :

```
calculerTotal()
verifierConnexion()
afficherProfil()
validerCode()
```

À éviter :

```
faire()
test()
fonction1()
abc()
```

En Swift, les noms de fonctions utilisent généralement le camelCase.

8. Les paramètres

Une fonction devient plus utile lorsqu'elle peut recevoir des données.

Exemple :

```
func afficherBonjour(nom: String) {
    print("Bonjour \(nom)")
}
```

Ici, nom: String est un paramètre.

9. Appeler une fonction avec un paramètre

```
func afficherBonjour(nom: String) {
    print("Bonjour \(nom)")
}
```

```
afficherBonjour(nom: "Alex")
```

Résultat :

```
Console
Bonjour Alex
```

Autre appel :

```
afficherBonjour(nom: "Marie")
```

Résultat :

```
Console
Bonjour Marie
```

La même fonction peut donc travailler avec différentes valeurs.

10. Paramètre et argument

Dans :

```
func afficherBonjour(nom: String) {
    print("Bonjour \(nom)")
}
```

nom est le paramètre.

Dans :

```
afficherBonjour(nom: "Alex")
```

"Alex" est l'argument envoyé à la fonction.

Terme	Signification
Paramètre	Variable définie dans la fonction
Argument	Valeur fournie lors de l'appel

11. Plusieurs paramètres

Une fonction peut recevoir plusieurs valeurs.

```
func afficherProfil(nom: String, age: Int) {
    print("Nom : \(nom)")
    print("Âge : \(age)")
}
```

Appel :

```
afficherProfil(nom: "Alex", age: 25)
```

Résultat :

```
Console
Nom : Alex
Âge : 25
```

12. Exemple — Informations sur un appareil

```swift
func afficherAppareil(modele: String, batterie: Int) {
    print("Appareil : \(modele)")
    print("Batterie : \(batterie)%")
}
```

Appel :

```swift
afficherAppareil(modele: "iPhone", batterie: 85)
```

Résultat :

```
Console
Appareil : iPhone
Batterie : 85%
```

13. Retourner une valeur

Certaines fonctions ne font pas seulement une action. Elles doivent calculer et retourner une valeur.

Exemple :

```swift
func additionner(a: Int, b: Int) -> Int {
    return a + b
}
```

La partie -> Int indique que la fonction retourne un entier.

14. Utiliser la valeur retournée

```swift
func additionner(a: Int, b: Int) -> Int {
    return a + b
}

let resultat = additionner(a: 10, b: 5)
print(resultat)
```

Résultat :

```
Console
15
```

La valeur retournée peut être stockée dans une variable ou une constante.

15. Exemple — Calculer un total

```swift
func calculerTotal(prix: Double, quantite: Int) -> Double {
    return prix * Double(quantite)
}
```

Utilisation :

```swift
let total = calculerTotal(prix: 12.50, quantite: 4)
print("Total : \(total) $")
```

16. return

return permet de renvoyer une valeur à l'endroit où la fonction a été appelée.

Exemple :

```swift
func calculerCarre(nombre: Int) -> Int {
    return nombre * nombre
}
```

Appel :

```swift
let resultat = calculerCarre(nombre: 5)
```

La fonction retourne :

```
Console
25
```

17. Retour implicite simple

Pour une fonction très courte contenant une seule expression, Swift permet parfois d'omettre return.

```swift
func calculerCarre(nombre: Int) -> Int {
    nombre * nombre
}
```

Cette version est équivalente à :

```swift
func calculerCarre(nombre: Int) -> Int {
    return nombre * nombre
}
```

Pour commencer, utiliser return explicitement peut toutefois rendre la lecture plus claire.

18. Fonction sans valeur de retour

Certaines fonctions exécutent simplement une action.

Exemple :

```
func afficherErreur() {
    print("Une erreur est survenue")
}
```

Il n'est pas nécessaire d'écrire -> Int ou return : la fonction ne retourne aucune donnée utile.

19. Fonction avec valeur de retour

```
func calculerMoyenne(note1: Double, note2: Double) -> Double {
    return (note1 + note2) / 2
}
```

Utilisation :

```
let moyenne = calculerMoyenne(note1: 75, note2: 85)
print("Moyenne : \(moyenne)")
```

20. Utiliser une condition dans une fonction

Une fonction peut contenir tout ce qui a déjà été vu :

- Variables
- Opérateurs
- Conditions
- Boucles

Exemple :

```
func verifierMajorite(age: Int) {
    if age >= 18 {
        print("Accès autorisé")
    } else {
        print("Accès refusé")
    }
}
```

Appel :

```
verifierMajorite(age: 21)
```

21. Retourner un résultat selon une condition

```
func estMajeur(age: Int) -> Bool {
    if age >= 18 {
        return true
    } else {
        return false
    }
}
```

Utilisation :

```
let resultat = estMajeur(age: 20)
print(resultat)
```

Résultat :

```
Console
true
```

22. Version simplifiée

La comparaison age >= 18 produit déjà un Bool. On peut donc écrire :

```
func estMajeur(age: Int) -> Bool {
    return age >= 18
}
```

Puis :

```
if estMajeur(age: 20) {
    print("Accès autorisé")
}
```

23. Exemple — Vérifier le niveau de batterie

```
func batterieFaible(niveau: Int) -> Bool {
    return niveau <= 20
}
```

Utilisation :

```
let batterie = 15
if batterieFaible(niveau: batterie) {
    print("Attention : batterie faible")
}
```

Cette approche permet de donner un nom clair à une règle métier.

Au lieu de répéter if batterie <= 20 partout dans le programme, on peut écrire if batterieFaible(niveau: batterie).

24. Les étiquettes d'arguments

Swift distingue :

- Le nom utilisé à l'extérieur de la fonction
- Le nom utilisé à l'intérieur

Exemple :

```
func envoyerMessage(a destinataire: String) {
    print("Message envoyé à \(destinataire)")
}
```

Appel :

```
envoyerMessage(a: "Alex")
```

Emplacement	Nom utilisé
À l'intérieur	destinataire
À l'extérieur	a

25. Pourquoi les étiquettes d'arguments ?

Elles permettent de rendre les appels plus naturels à lire.

Exemple :

```
func calculerDistance(de depart: Double, a arrivee: Double) -> Double {
    return arrivee - depart
}
```

Appel :

```
let distance = calculerDistance(de: 10, a: 35)
```

On peut lire presque comme une phrase : calculer la distance de 10 à 35.

C'est une particularité importante de la syntaxe Swift.

26. Supprimer l'étiquette externe

Le symbole _ permet de supprimer l'étiquette lors de l'appel.

```
func doubler(_ nombre: Int) -> Int {
    return nombre * 2
}
```

Appel :

```
let resultat = doubler(5)
```

et non :

```
doubler(nombre: 5)
```

27. Comparaison

Avec étiquette :

```
func afficherNom(nom: String) {
    print(nom)
}

afficherNom(nom: "Alex")
```

Sans étiquette :

```
func afficherNom(_ nom: String) {
    print(nom)
}

afficherNom("Alex")
```

Les deux approches sont possibles. Le choix dépend surtout de la lisibilité de l'appel.

28. Plusieurs étiquettes d'arguments

```
func calculerPrix(
    de prixUnitaire: Double,
    pour quantite: Int
) -> Double {
    return prixUnitaire * Double(quantite)
}
```

Appel :

```
let total = calculerPrix(de: 15.0, pour: 3)
```

Emplacement	Noms utilisés
À l'intérieur	prixUnitaire, quantite
À l'extérieur	de, pour

29. Paramètres avec valeur par défaut

Swift permet de donner une valeur par défaut à un paramètre.

Exemple :

```
func afficherMessage(
    texte: String,
    prefixe: String = "INFO"
) {
    print("[\(prefixe)] \(texte)")
}
```

Appel sans préciser prefixe :

```
afficherMessage(texte: "Application démarrée")
```

Résultat :

```
Console
[INFO] Application démarrée
```

30. Modifier la valeur par défaut

On peut toujours fournir une autre valeur :

```
afficherMessage(
    texte: "Connexion impossible",
    prefixe: "ERREUR"
)
```

Résultat :

```
Console
[ERREUR] Connexion impossible
```

Les paramètres par défaut permettent d'éviter de créer plusieurs fonctions presque identiques.

31. Exemple — Notification

```
func afficherNotification(
    message: String,
    priorite: String = "Normale"
) {
    print("Notification : \(message)")
    print("Priorité : \(priorite)")
}
```

Appel :

```
afficherNotification(message: "Nouveau message")
```

ou :

```
afficherNotification(
    message: "Batterie critique",
    priorite: "Élevée"
)
```

32. Portée des variables

Une variable déclarée à l'intérieur d'une fonction existe uniquement dans cette fonction.

```
func calculerTotal() {
    let total = 100
    print(total)
}
```

Ici, total est une variable locale.

En dehors de la fonction, print(total) provoquera une erreur.

33. Variable locale

```
func afficherConnexion() {
    let message = "Connexion réussie"
    print(message)
}
```

message existe uniquement entre les accolades de cette fonction.

On parle de portée locale.

34. Les paramètres sont aussi locaux

```
func afficherUtilisateur(nom: String) {
    print(nom)
}
```

Le paramètre nom n'existe que pendant l'exécution de cette fonction.

Cela permet à chaque fonction de travailler avec ses propres données sans polluer inutilement le reste du programme.

35. Exemple complet — Calcul d'une commande

```
func calculerSousTotal(
    prixUnitaire: Double,
    quantite: Int
) -> Double {
    return prixUnitaire * Double(quantite)
}

func calculerReduction(
    sousTotal: Double,
    estMembre: Bool
) -> Double {
    if estMembre {
        return sousTotal * 0.05
```

```
    } else {
        return 0
    }
}

let sousTotal = calculerSousTotal(
    prixUnitaire: 25,
    quantite: 4
)

let reduction = calculerReduction(
    sousTotal: sousTotal,
    estMembre: true
)

let totalFinal = sousTotal - reduction

print("Sous-total : \(sousTotal) $")
print("Réduction : \(reduction) $")
print("Total : \(totalFinal) $")
```

Ici, chaque fonction possède une responsabilité précise :

```
calculerSousTotal()
        ↓
calculerReduction()
        ↓
calcul du total final
```

36. Pourquoi découper un programme en fonctions ?

Considérons un programme qui doit :

- Valider une quantité
- Calculer un sous-total
- Calculer une réduction
- Calculer les taxes
- Afficher un résultat

On pourrait tout écrire dans un seul bloc. Mais une meilleure organisation serait :

```
validerQuantite(...)
calculerSousTotal(...)
calculerReduction(...)
calculerTaxes(...)
afficherResume(...)
```

Avantages :

- Code plus lisible
- Code plus facile à tester
- Moins de répétition
- Correction plus simple
- Réutilisation des traitements
- Responsabilités mieux séparées

37. Une fonction = une responsabilité

Une bonne pratique consiste à éviter les fonctions qui font trop de choses.

À éviter conceptuellement :

```
func toutFaire() {
    // validation
    // calcul
    // connexion
    // affichage
    // sauvegarde
    // ...
}
```

Préférable :

```
func validerDonnees() {
}

func calculerTotal() {
}

func afficherResultat() {
}
```

L'objectif est de créer des fonctions faciles à comprendre et à réutiliser.

38. Exemple - Calculateur de batterie

Créer une fonction :

```
func obtenirEtatBatterie(niveau: Int) -> String
```

Règles :

Niveau	État
0 à 20	Critique
21 à 50	Faible

51 à 80	Normal
81 à 100	Excellent

Une solution possible :

```
func obtenirEtatBatterie(niveau: Int) -> String {
    if niveau <= 20 {
        return "Critique"
    } else if niveau <= 50 {
        return "Faible"
    } else if niveau <= 80 {
        return "Normal"
    } else {
        return "Excellent"
    }
}
```

Utilisation :

```
let niveau = 65
let etat = obtenirEtatBatterie(niveau: niveau)
print("Batterie : \(niveau)%")
print("État : \(etat)")
```

39. Exemple guidé — Validation d'un code

```
func codeValide(
    codeSaisi: Int,
    codeCorrect: Int = 1234
) -> Bool {
    return codeSaisi == codeCorrect
}
```

Utilisation :

```
let code = 1234
if codeValide(codeSaisi: code) {
    print("Accès autorisé")
} else {
    print("Code incorrect")
}
```

On combine ici : fonction + paramètres + valeur par défaut + Bool + condition.

40. Exercice - Gestion simplifiée d'une batterie

Créer les fonctions suivantes.

Fonction 1

```
func estNiveauValide(_ niveau: Int) -> Bool
```

Elle doit retourner true uniquement si le niveau est compris entre 0 et 100.

Fonction 2

```
func obtenirEtatBatterie(niveau: Int) -> String
```

Règles :

Niveau	État
0 à 20	"Critique"
21 à 50	"Faible"
51 à 80	"Normal"
81 à 100	"Excellent"

Fonction 3

```
func afficherBatterie(
    niveau: Int,
    appareil: String = "iPhone"
)
```

Elle doit afficher un résumé ressemblant à :

```
Console
Appareil : iPhone
Batterie : 72%
État : Normal
```

Avant d'afficher le résultat, vérifier que le niveau est valide. Si le niveau est invalide, afficher :

```
Console
Erreur : niveau de batterie invalide
```

Éléments requis :

- Au moins trois fonctions
- Des paramètres
- Une valeur de retour Bool
- Une valeur de retour String
- Un paramètre avec valeur par défaut
- Au moins un paramètre sans étiquette externe avec _
- Des conditions
- L'interpolation de chaînes

41. Exercice - Calcul d'un téléchargement

Créer une fonction :

```
func calculerPourcentage(
    telecharge: Double,
    total: Double
) -> Double
```

Elle doit retourner le pourcentage téléchargé.

Exemple :

```
let progression = calculerPourcentage(
    telecharge: 250,
    total: 1000
)
```

Résultat :

```
Console
25.0
```

Puis créer une deuxième fonction :

```
func afficherProgression(pourcentage: Double)
```

qui affiche :

```
Console
Téléchargement : 25.0%
```

Le but est de séparer le calcul de l'affichage.

42. À retenir

Une fonction est déclarée avec :

```
func
```

Exemple :

```
func afficherBonjour() {
    print("Bonjour")
}
```

Une fonction peut recevoir des paramètres :

```
func afficherBonjour(nom: String) {
}
```

Elle peut retourner une valeur :

```
func additionner(a: Int, b: Int) -> Int {
    return a + b
}
```

Swift permet de contrôler la façon dont les paramètres sont écrits lors de l'appel :

```swift
func doubler(_ nombre: Int) -> Int
```

ou :

```swift
func envoyerMessage(a destinataire: String)
```

Les paramètres peuvent également avoir des valeurs par défaut :

```swift
func afficherMessage(
    texte: String,
    priorite: String = "Normale"
)
```

43. Vision globale

Les fonctions permettent maintenant d'organiser les notions déjà acquises :

```
Variables
    ↓
Opérateurs
    ↓
Conditions
    ↓
Boucles
    ↓
Fonctions
```

Au lieu d'écrire un programme comme une longue suite d'instructions, on peut commencer à le découper en traitements nommés, réutilisables et responsables d'une tâche précise.

À venir — Partie 2 : retour de plusieurs valeurs avec les tuples, fonctions comme valeurs et paramètres, types de fonctions et fonctions imbriquées, avant d'aborder plus tard les closures.